



12.1. Heaps

"Keep the most important element where you can always find it."

Previous Section:

 [12. Priority Queues](#)

Contents:

[1. What is a Heap?](#)

[1.1. Min Heap](#)

[1.2. Max Heap](#)

[2. Implementation of Heaps](#)

[3. Applications of Heaps](#)

[Advantages of Heaps](#)

[Disadvantages of Heaps](#)

[Summary](#)

[References](#)

In the previous section, we learned about Priority Queues. A Priority Queue always needs a fast way to answer a simple question: ***"Which element has the highest priority right now?"***

For a small dataset, we could simply search through every element. But what if we have thousands, millions, or even billions of elements? Searching through the entire collection every time would be extremely inefficient.

This is where **Heaps** enter the story. A Heap is a special tree structure designed to keep the most important element at the top, allowing us to access it quickly while still supporting efficient insertion and deletion.

Think of a hospital waiting system. Instead of searching through every patient's record whenever a doctor becomes available, we organize the patients in a special structure where the highest-priority patient is always positioned at the top.

The result?

- Find highest priority patient → Immediate
- Insert new patient → Efficient
- Remove treated patient → Efficient

This simple idea makes heaps one of the most important data structures in Computer Science.

1. What is a Heap?

A **Heap** is a special type of **Complete Binary Tree**. Unlike an ordinary binary tree, nodes are inserted following **Level Order Traversal**. This means:

- Levels are filled from top to bottom.
- Within each level, nodes are filled from left to right.

Since every level is completely filled before moving to the next level, the tree is compact and easy to store inside an array.

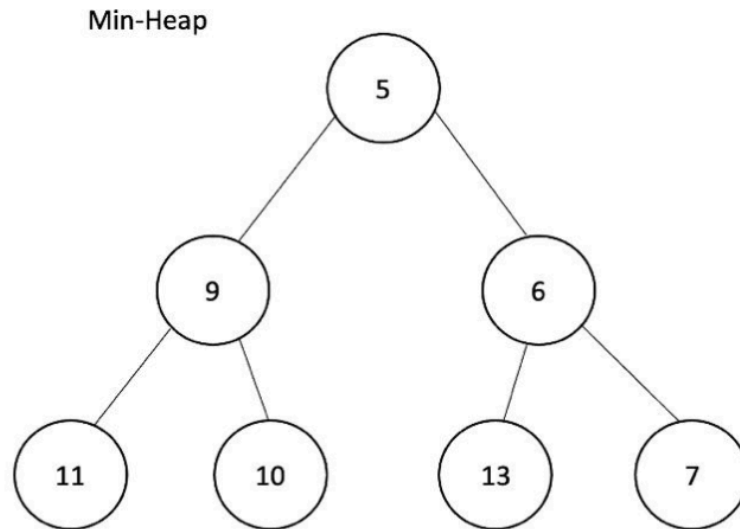
Common Types of Heaps

Many heap variations exist: Binary Heap; Binomial Heap; Fibonacci Heap; Skew Heap; Leftist Heap. In this section, we focus on the most common type: **Binary Heap**.

Binary Heaps are divided into two categories: ***Min Heap and Max Heap***

1.1. Min Heap

A **Min Heap** ensures that every parent node contains a value smaller than or equal to its children.



Observe:

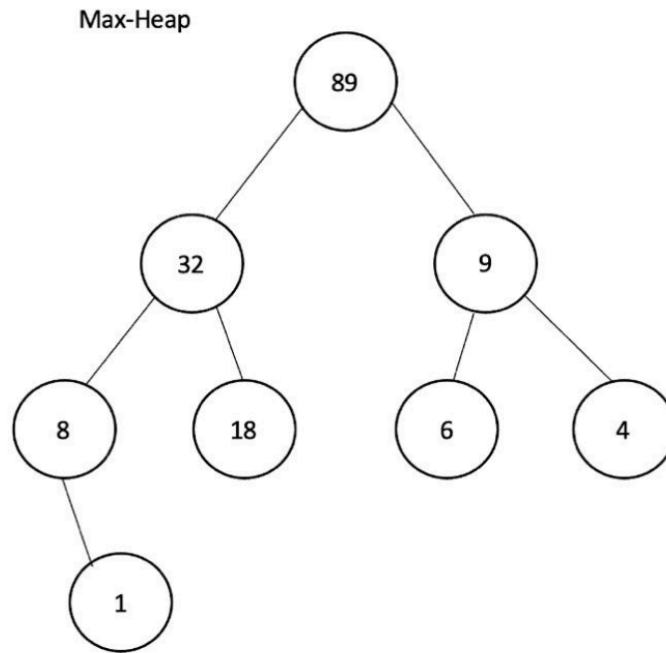
- $5 \leq 6, 9$
- $6 \leq 7, 13$
- $9 \leq 10, 11$

The smallest value always remains at the root. Therefore: **Root = Minimum Element**

When removing an element from a Min Heap: **Removed first $\rightarrow$ 5**

1.2. Max Heap

A **Max Heap** ensures that every parent node contains a value greater than or equal to its children.



Observe:

- $89 \geq 9, 32$
- $9 \geq 4, 6$
- $32 \geq 8, 18$
- $8 \geq 1$

The largest value always remains at the root. Therefore: **Root = Maximum Element**

When removing an element from a Max Heap: **Removed first $\rightarrow$ 89**

Why the Root Matters

The root node is special because all removals happen from the root.

- For a Max Heap: **Root = Largest Element**
- For a Min Heap: **Root = Smallest Element**

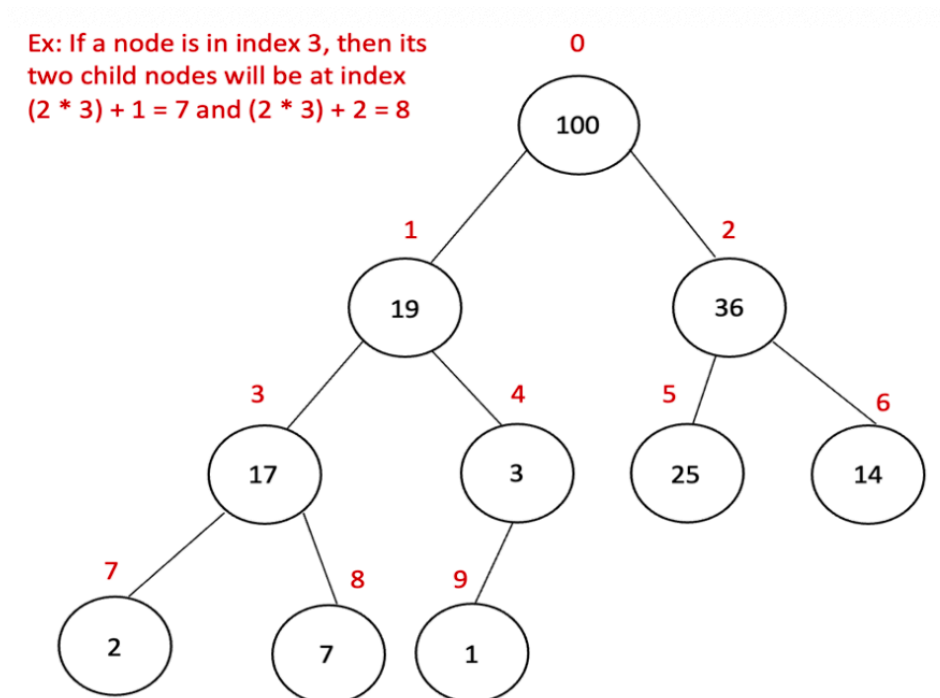
This is exactly why heaps are perfect for Priority Queues. The highest-priority element is always available immediately.

2. Implementation of Heaps

One of the most beautiful aspects of heaps is that they do not require explicit tree nodes.

Since a heap is always a Complete Binary Tree, it can be stored efficiently using an array.

Consider this heap:



Instead of storing links between nodes, we store the values in level order:

```
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 100 | 19 | 36 | 17 | 3 | 25 | 14 | 2 | 7 | 1 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
```

Parent and Child Relationships

Suppose a node is stored at index i . Then:

- **Left Child:** $2i + 1$; **Right Child:** $2i + 2$.
 - Example: Index = 1 → Value = 19 . Children: Left Child = $2(1)+1 = 3$; Right Child = $2(1)+2 = 4$. Which corresponds to: 17 and 3
- Parent Node: For any node (except the root): Parent = $(i - 1) / 2$

- Example: `Index = 8 → Value = 7` . Parent: `(8 - 1)/2 = 3` . Index 3 contains: 17.
Therefore: `Parent of 7 = 17`

Why Arrays Work So Well

A complete binary tree contains no gaps. Therefore: No wasted memory. No extra pointers. Fast access. Simple implementation

This is one of the main reasons heaps are so widely used in real-world systems.

3. Applications of Heaps

At first glance, heaps may seem like a specialized structure. In reality, they appear everywhere.

- Priority Queues: The most common application.

Highest Priority
↓
[Root]

- Used in: Hospitals; Task management; Operating systems; Network scheduling
- Heap Sort: Heap Sort is built directly upon heap operations.
 - Build Heap → Remove Root → Rebuild Heap → Repeat
 - Because of its predictable performance, Heap Sort is efficient for large datasets.
- Graph Algorithms: Many famous graph algorithms rely on heaps. Examples: Dijkstra's Algorithm; Prim's Algorithm; A* Search Algorithm
 - These algorithms repeatedly need the smallest or highest-priority element. A heap provides exactly that.
- Lossless Compression: Huffman Coding uses Min Heaps to build Huffman Trees.

- Smallest Frequencies → Combined First.
- This helps produce efficient compression schemes.

- Medical Applications: Just like the hospital example in the Priority Queues section.
 - Critical Patient → High Priority → Normal Patient → Lower Priority
 - The most urgent patients can always be processed first.

- **Load Balancing:** Servers may have different workloads.
 - Server A → 15 Tasks
 - Server B → 4 Tasks
 - Server C → 2 Tasks
 - A heap quickly identifies: Lowest Load Server, allowing tasks to be distributed efficiently.

- **Resource Allocation:** Resources such as CPU Time; Memory Blocks; Storage Requests can all be managed using heaps. The highest-priority requests are processed first.

- **Job Scheduling:** Modern systems continuously schedule jobs.
 - Priority 1 → Emergency Task
 - Priority 2 → System Task
 - Priority 3 → User Task
 - A heap ensures that the most important task is always available immediately.

Advantages of Heaps

- Fast Insertion and Deletion: Both operations require only $O(\log n)$, making heaps suitable for large datasets.

- Fast Access to Minimum or Maximum Element with $O(1)$. The root always stores the most important value.
- Memory Efficient: Since heaps are complete binary trees, stored using arrays and no wasted space
- Dynamic Structure: Elements can be added or removed at runtime. without rebuilding the entire structure.
- Excellent for Priority-Based Processing: Ideal for Medical systems; Load balancing; Job scheduling; Stock market analysis; Operating systems

Disadvantages of Heaps

Like every data structure, heaps are powerful but not perfect.

- **Limited Flexibility:** A heap focuses on maintaining one specific ordering rule (`Parent  $\geq$  Children` or `Parent  $\leq$  Children`). This makes it less flexible than many other structures.
- **Poor Searching Performance:** Suppose we want to find `Value = 47` . A heap provides no efficient searching mechanism. In the worst case, we must inspect every node with $O(n)$
- Not Stable: Two equal values may change their relative order.
 - `5A - 5B` $\rightarrow$ After heap operations `5B - 5A` .
 - The original ordering is not guaranteed.
- Not Always the Fastest Choice: Although heaps are efficient with $O(\log n)$ for insertion and deletion, some specialized algorithms may outperform heaps depending on the problem.

Summary

A **Heap** is a special type of Complete Binary Tree that maintains a priority relationship between parent and child nodes. In a **Min Heap**, the smallest element is always at the root, while in a **Max Heap**, the largest element remains at the root. Because of this property, heaps provide efficient insertion and deletion operations and are commonly used to implement **Priority Queues**.

One of the biggest advantages of heaps is that they can be stored compactly in an array without wasting memory. As a result, heaps are widely used in applications such as Heap Sort, graph algorithms, job scheduling, load balancing, resource allocation, data compression, and medical systems. While heaps excel at priority-based processing, they are less suitable for searching specific elements and do not preserve the relative order of equal values.

Most importantly, heaps serve as the foundation of Priority Queues, allowing systems to quickly access the most important element whenever needed.

In the next section, we will move beyond the theory and learn how heaps actually maintain their structure through **Heap Insertion**, **Heap Deletion**, and the fascinating process known as **Heapify**, which allows the heap to continuously preserve its ordering properties after every modification.

References

https://drive.google.com/file/d/1VfNggFPkvnDGZeVKn_QqIHjWCDQRapBM/view?usp=drive_link

<https://www.geeksforgeeks.org/dsa/heap-data-structure/>

<https://www.programiz.com/dsa/heap-data-structure>

<https://www.youtube.com/watch?v=HqPJF2L5h9U&t=2141s>

Next Section:

 [12.2. Operations on Heaps](#)